

武汉理工大学

课 程 名 称	企业级应用软件设计与开发
项 目 名 称	基于 LangGraph 的多智能体 动漫推荐系统
方 向	方向一：Agentic AI 原生开 发
学 号	2025302926
学 生 姓 名	刘锦昊
专 业	计算机技术
指 导 教 师	戚欣
提 交 日 期	2026 年 6 月 22 日

目录

基于 LangGraph 的多智能体动漫推荐系统课程大报告1

一、选题背景与设计思想1

1.1 选题背景1

1.2 问题定义1

1.3 现有方案不足2

1.4 项目价值2

1.5 技术路线3

二、Specs 规格文档5

2.1 Product Spec: 产品规格5

2.2 Architecture Spec: 架构规格7

2.3 API Spec: 接口规格8

三、系统架构与设计12

3.1 核心架构设计12

3.2 Agent 交互流程12

3.3 数据流设计14

四、关键实现与代码展示16

4.1 LangGraph 核心循环设计16

4.2 状态设计 AgentState18

4.3 意图理解实现19

4.4 Bangumi 工具定义20

4.5 数据过滤实现20

4.6 Tavily 多维搜索实现22

4.7 推荐生成实现22

4.8 质量评估与反思循环23

4.9 配置文件设计23

4.10 AI IDE 使用说明24

五、测试与评估25

5.1 功能测试25

5.2 测试结果分析25

5.3 Agent 行为评估26

5.4 Demo 展示设计26

六、系统升级与扩展34

6.1 可扩展架构34

6.2 下一阶段计划34

6.3 AI 能力演进路径35

七、课程总结36

7.1 个人收获36

7.2 工程思维转变36

7.3 对课程的建议37

7.4 总结37

基于 LangGraph 的多智能体动漫推荐系统课程大报告

一、选题背景与设计思想

1.1 选题背景

随着在线内容平台的发展，用户面对的动漫作品数量越来越多。传统的动漫检索方式通常依赖关键词、标签、排行榜或人工编辑推荐，例如用户输入“恋爱”“异世界”“高分番剧”等关键词后，系统返回一批相关条目。这类方式虽然实现简单，但存在明显局限：第一，用户的自然语言需求往往比较模糊，例如“想看轻松一点的恋爱喜剧”“最近比较火的异世界番”“类似某部作品但不要太沉重”，传统关键词检索很难准确理解其中包含的情绪、风格、年代、评分和排除条件；第二，单一数据库检索只能获取静态条目数据，无法充分结合网络口碑、近期热度和用户评价；第三，传统推荐系统一般缺少自我检查机制，可能出现推荐结果与用户需求不匹配、同一系列重复推荐、低质量条目混入等问题。

本项目选择“动漫推荐”作为应用场景，设计并实现了一个基于 LangGraph 的多智能体动漫推荐系统。系统使用 DeepSeek Chat API 作为大模型能力来源，结合 Bangumi 动漫数据库和 Tavily 网络搜索工具，通过 Streamlit 提供可交互的 Web 聊天界面。用户只需要用中文描述想看的动漫类型，系统即可完成意图理解、数据检索、结果筛选、推荐生成、质量评估和反思优化等流程。

1.2 问题定义

本项目要解决的问题可以定义为：

在用户以自然语言输入动漫推荐需求的情况下，系统如何自动理解用户意图，调用外部动漫数据库和网络搜索工具，筛选符合条件的候选作品，并生成结构化、可解释、质量可控的动漫推荐结果。

具体来说，系统需要完成以下任务：

从用户自然语言中提取核心搜索词、偏好标签、过滤条件和是否需要网络搜索。

根据意图调用 Bangumi 数据库接口获取候选动漫条目。

在涉及“最新”“热门”“网络口碑”“今年”等时效性需求时,额外调用 Tavily 网络搜索获取补充信息。

对候选数据进行低质量过滤、类型过滤、年代过滤、评分过滤和标签过滤。

使用大模型从候选池中生成推荐结果,并给出推荐理由、匹配点、评分、标签、封面链接和 Bangumi 条目链接。

引入 Critic 质量评估节点,对推荐结果进行自动审查,如果不合格则进行反思优化和重新生成。

通过 Streamlit 页面展示推荐结果、筛选条件、用户偏好和系统执行过程。

1.3 现有方案不足

现有的简单动漫推荐方案主要包括三类:

第一类是关键词检索方案。用户输入关键词后,系统在数据库中直接搜索相关作品。这类方案实现简单,但无法理解复杂语义。例如用户输入“不要太虐的恋爱番”,系统可能只识别“恋爱番”,无法正确处理“不要太虐”的偏好约束。

第二类是基于标签或评分的静态推荐方案。系统按照标签、评分、收藏数进行排序,然后返回结果。这类方案可解释性较强,但缺少自然语言理解能力,也无法根据用户临时需求进行动态组合推荐。

第三类是直接使用大模型生成推荐的方案。大模型可以理解自然语言并生成比较自然的推荐理由,但容易产生幻觉,例如推荐不存在的作品、引用错误评分、忽略用户筛选条件,或者推荐结果没有可靠数据来源。

本项目针对上述不足进行了改进:一方面利用大模型完成意图理解和推荐理由生成,另一方面通过 Bangumi 数据库提供结构化数据约束,通过 Tavily 搜索补充时效信息,并通过 Critic 反思循环检查推荐质量,从而兼顾自然语言理解、数据可靠性、结果可解释性和推荐质量控制。

1.4 项目价值

本项目的价值主要体现在以下几个方面。

首先,它体现了 Agent 系统的工程化设计思想。系统不是简单调用一次大

模型，而是将推荐任务拆分为多个节点，包括意图理解、数据库检索、网络搜索、候选合并、推荐生成、质量评估、搜索优化和结果格式化。每个节点承担明确职责，整体流程由 LangGraph 进行编排。

其次，它提高了推荐结果的可靠性。系统推荐的作品来自 Bangumi 数据库，包含作品名称、评分、标签、简介、收藏数和封面链接等结构化字段，减少了大模型直接编造结果的风险。

再次，它提高了推荐过程的可解释性。系统不仅输出推荐作品，还输出推荐理由、匹配点、执行路径和过程日志，用户可以看到系统为什么推荐这些作品。

最后，它具有良好的扩展性。未来可以接入更多动漫数据库、用户画像数据库、向量检索系统、知识图谱或多模态封面理解模块，将其扩展成更加完整的智能内容推荐平台。

1.5 技术路线

本项目整体技术路线如下：

用户在 Streamlit 聊天界面输入需求后，系统首先进入 LangGraph workflow。意图理解 Agent 调用 DeepSeek 模型，将自然语言转换为搜索关键词、浏览标签、偏好信息和过滤条件。随后系统根据是否需要时效性信息决定是否调用 Tavily 搜索。如果需要，系统先进行网络搜索并总结口碑信息；无论是否调用 Tavily，系统都会调用 Bangumi API 获取候选动漫数据。候选数据经过低质量过滤、评分过滤、类型过滤、年代过滤和标签过滤后，进入推荐生成 Agent。推荐生成 Agent 使用大模型结合候选数据和用户偏好生成结构化推荐列表。之后 Critic Agent 对推荐结果进行审查，如果结果不满足要求，则进入反思优化节点，调整搜索策略或清洗候选池，再重新生成推荐。最终系统将通过审查或兜底生成的推荐结果格式化，并在前端以卡片形式展示。

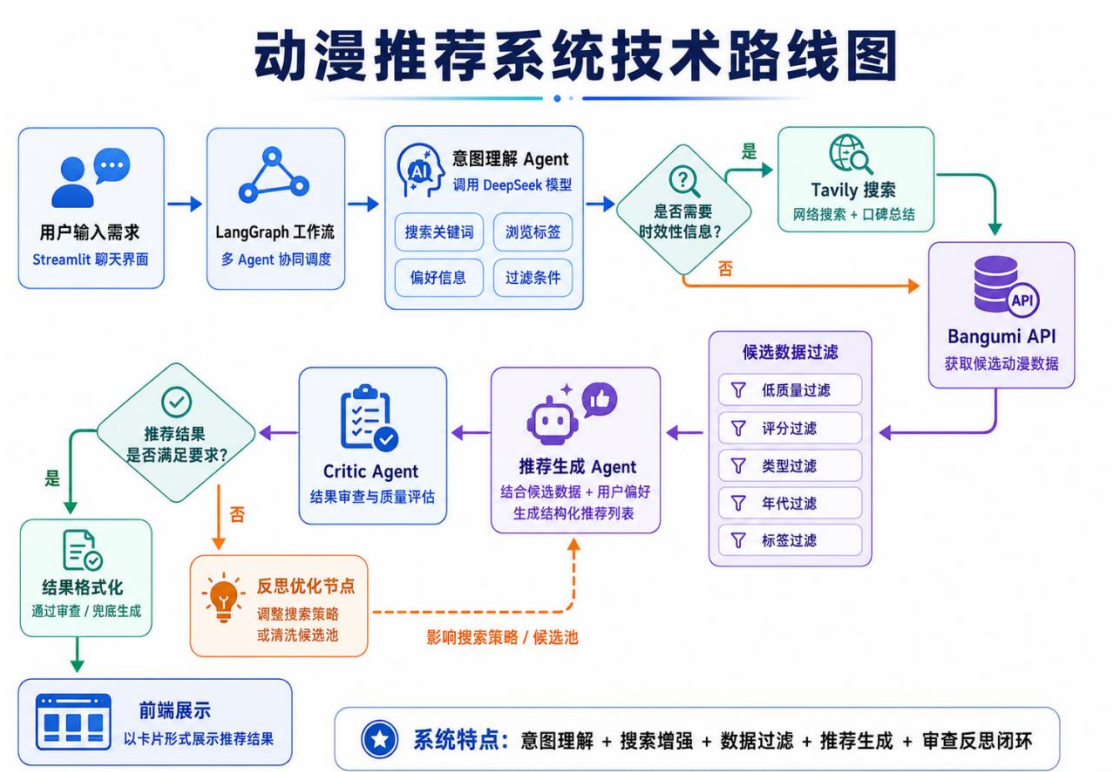


图 1.1：项目整体技术路线图

二、Specs 规格文档

本部分从 Product Spec、Architecture Spec 和 API Spec 三个角度描述系统规格。

2.1 Product Spec: 产品规格

2.1.1 产品名称

基于 LangGraph 的多智能体动漫推荐系统。

2.1.2 目标用户

本系统面向希望快速获得动漫推荐的普通用户，也适合对 Agent workflows、多工具调用和大模型推荐系统感兴趣的学习者。用户不需要了解 Bangumi、Tavily 或大模型调用方式，只需要输入自然语言需求即可获得推荐结果。

2.1.3 核心功能

系统核心功能包括：

自然语言推荐

用户可以直接输入“推荐几部高分异世界番”“想看轻松治愈的日常番”“最近比较热门的恋爱喜剧”等自然语言需求。

多智能体协作

系统将任务拆分为多个 Agent 节点，包括意图理解、数据库搜索、网络搜索、推荐生成、质量评估和反思优化。

Bangumi 数据库检索

系统通过 Bangumi API 获取动漫条目数据，包括作品名称、中文名、评分、标签、简介、集数、播出日期、收藏数和封面链接。

Tavily 网络搜索

当用户需求中包含“最新”“今年”“热门”“网络口碑”“评价”等时效性或外部评价相关表达时，系统调用 Tavily 进行补充搜索。

硬性筛选

系统支持按照动漫类型、播出年代、最低评分和标签进行筛选。

反思循环

推荐结果生成后，Critic Agent 会检查推荐结果是否符合用户需求。如果不符合，

系统会进行搜索策略优化或重新生成。

可视化界面

系统使用 Streamlit 构建 Web 界面，支持聊天输入、推荐卡片展示、筛选面板、偏好摘要和推荐过程日志查看。

2.1.4 输入规格

系统输入主要包括两类：

第一类是用户自然语言输入，例如：

“推荐几部高分恋爱喜剧，不要太虐。”

“最近比较火的异世界番有哪些？”

“想看轻松治愈一点的日常番。”

第二类是侧边栏筛选条件，包括：

动漫类型：TV、剧场版、OVA、WEB、其他。

播出年代：例如 2000—2026。

最低评分：0.0—10.0。

标签/风格：治愈、热血、悬疑、异世界、恋爱、科幻、搞笑、奇幻、战斗、校园、日常、美食、音乐、运动、推理等。

2.1.5 输出规格

系统输出为结构化推荐结果，每部作品包含：

作品中文名。

作品原名。

推荐理由。

匹配点。

Bangumi 评分。

标签列表。

封面图片。

Bangumi 条目链接。

此外，系统还输出推荐分析过程，包括意图分析、Tavily 搜索、Bangumi 搜索、低质量过滤、硬性筛选、候选池构建、AI 推荐生成、质量审查和最终输出等步骤。

2.1.6 非功能需求

可用性

用户通过浏览器访问 `Streamlit` 页面即可使用系统，不需要手动调用 `API`。

可解释性

系统需要展示推荐理由和匹配点，并保留推荐过程日志。

鲁棒性

当大模型输出无法解析、搜索结果为空或质量评估失败时，系统需要有回退策略和兜底推荐机制。

可扩展性

系统架构应支持未来添加新工具、新数据源、新 `Agent` 节点和新评估指标。

安全性

`API Key` 通过 `.env` 文件配置，不直接写死在代码中。

2.2 Architecture Spec: 架构规格

2.2.1 系统分层

系统可以分为四层：

第一层是用户交互层。该层由 `Streamlit` 实现，负责接收用户输入、展示筛选条件、显示推荐结果和过程日志。

第二层是 `Agent` 编排层。该层由 `LangGraph` 实现，负责定义节点、边、条件分支和反思循环。

第三层是智能决策层。该层由 `DeepSeek Chat API` 提供大模型能力，主要用于意图理解、推荐生成、质量审查和搜索策略优化。

第四层是外部工具与数据层。该层包括 `Bangumi API`、`Tavily Search API` 和本地配置文件。

2.2.2 核心模块

`app.py`

系统入口文件，负责 `Streamlit` 页面配置、会话状态管理、筛选面板调用、用户输入处理、`LangGraph` 调用和结果展示。

`graph.py`

workflow编排文件，负责创建 `LangGraph` 状态图，定义节点和条件边。

agents.py

核心 Agent 逻辑文件，包含意图理解、Tavily 搜索节点、Bangumi 拉取节点、数据合并、推荐生成、质量评估、搜索优化和结果格式化等功能。

tools.py

工具封装文件，负责 Bangumi API 调用、Tavily API 调用、数据清洗、类型识别、筛选和搜索结果总结。

state.py

状态定义文件，使用 TypedDict 定义 AgentState，包括用户输入、偏好、过滤参数、搜索关键词、候选数据、推荐结果、评估反馈、反思次数、执行路径和过程日志。

ui_components.py

界面组件文件，负责推荐卡片、筛选面板、偏好摘要、聊天消息和流程日志展示。

2.2.3 Agent workflow

系统 workflow 可以描述为：

开始节点进入 `understand_intent`，系统识别用户意图。如果判断需要网络搜索，并且 Tavily 调用次数未超过限制，则进入 `search_tavily`，之后再进入 `fetch_bangumi`；否则直接进入 `fetch_bangumi`。接着系统进入 `merge_data` 合并数据，然后进入 `generate_recommendations` 生成候选推荐。推荐结果进入 `evaluate_recommendations` 质量审查。如果审查通过，则进入 `format_response` 输出结果；如果审查不通过且反思次数小于 2，则进入 `refine_search` 优化搜索策略。优化后如果需要重新拉取数据，则回到 `fetch_bangumi`；否则回到 `generate_recommendations`。最终由 `format_response` 输出最终推荐，并结束流程。

2.3 API Spec：接口规格

2.3.1 DeepSeek Chat API

用途：提供大模型推理能力。

主要调用场景：

意图理解：将用户自然语言转换为结构化 JSON。

推荐生成：从候选动漫中选择符合需求的作品，并生成推荐理由。

质量审查：判断推荐列表是否满足用户需求。

搜索优化：根据评估反馈调整搜索关键词。

配置方式：

DEEPSEEK_API_KEY=你的 DeepSeek API Key

模型配置：

```
model = "deepseek-chat"
```

```
base_url = "https://api.deepseek.com/v1"
```

```
temperature = 0.7
```

2.3.2 Bangumi API

用途：获取动漫条目数据。

主要接口类型：

关键词搜索接口

根据关键词搜索动漫作品。

条目详情接口

根据 `subject_id` 获取作品详细信息。

条目浏览/筛选接口

根据标签、评分、年代等条件浏览动漫条目。

主要返回字段：

`id`

`name`

`name_cn`

`images_large`

`summary`

`rating_score`

`rating_total`

`tags`

`date`

`air_date`

`eps`

item_type

collect_count

2.3.3 Tavily Search API

用途：补充网络口碑、近期评价、热门推荐和外部信息。

触发条件：

当用户查询中包含“最新”“最近”“刚完结”“网络口碑”“今年”“评价”“推荐排行”“热门”等表达时，系统判断需要调用 Tavily。

配置方式：

TAVILY_API_KEY=你的 Tavily API Key

搜索策略：

系统会根据用户搜索关键词、偏好标签和筛选参数构建多条搜索语句，例如：

{关键词} 动漫 推荐 最佳

{关键词} 动漫 评价 口碑

{关键词} anime review recommendation rating

{关键词} anime 人气排名 热门推荐 2024 2025

2.3.4 Streamlit 前端接口

用途：提供用户交互界面。

主要交互入口：

st.chat_input("描述你想看的动漫类型...")

主要界面组件：

聊天消息展示。

推荐卡片展示。

筛选条件侧边栏。

偏好摘要展示。

推荐流程日志展示。

2.3.5 内部状态接口 AgentState

系统通过 AgentState 在节点之间传递状态。核心字段包括：

user_query: 用户原始查询

preferences: 用户偏好

filter_params: 筛选参数

search_keywords: 搜索关键词
browse_tags: 浏览标签
need_tavily: 是否需要 Tavily
tavily_results: Tavily 搜索摘要
bangumi_data: Bangumi 候选数据
candidate_recommendations: 候选推荐结果
final_recommendations: 最终推荐结果
reflection_count: 反思次数
pass_evaluation: 是否通过质量审查
evaluation_feedback: 审查反馈
need_refetch: 是否需要重新拉取
execution_path: 执行路径
process_log: 过程日志

三、系统架构与设计

3.1 核心架构设计

系统采用“前端交互 + Agent 编排 + 工具调用 + 外部数据源”的整体架构。

用户在前端输入需求后，Streamlit 将用户输入和筛选参数封装为初始状态。LangGraph 负责调度各个 Agent 节点。DeepSeek 负责自然语言理解和生成任务。Bangumi API 负责提供可靠的动漫条目数据。Tavily API 负责提供时效性和网络口碑信息。最后系统将推荐结果返回前端，以卡片形式展示。



图 3.1：核心架构图

系统架构的核心特点是解耦。前端不直接处理推荐逻辑，只负责展示和会话管理；Agent 节点不直接处理界面，只负责状态转换；工具函数不参与高层推理，只负责外部 API 调用和数据清洗。这种设计降低了模块之间的耦合度，便于后续扩展和调试。

3.2 Agent 交互流程

系统中各 Agent 的职责如下：

意图理解 Agent

输入用户查询和已有筛选条件，输出搜索关键词、浏览标签、偏好信息、是否需要 Tavily 以及更新后的筛选参数。该节点解决的是“用户到底想要什么”的问题。

Tavily 搜索 Agent

当用户需求涉及最新信息、热门趋势或网络评价时，该节点调用 Tavily 进行多维度搜索，并将搜索结果总结为文本。

Bangumi 检索 Agent

该节点根据搜索关键词、标签和筛选参数调用 Bangumi API，获取候选动漫数据，并进行去重、低质量过滤和硬性筛选。

数据合并节点

该节点将 Bangumi 数据和 Tavily 网络评价进行逻辑合并，为推荐生成提供上下文。

推荐生成 Agent

该节点根据用户偏好、候选池和网络评价生成推荐结果。输出格式为 JSON 数组，每部作品包含标题、理由、匹配点、评分、标签、封面和 Bangumi ID。

质量评估 Agent

该节点扮演 Critic 角色，检查推荐结果是否存在同系列重复、核心类型不匹配、无效 ID、推荐理由不具体等问题。

搜索优化 Agent

当质量评估不通过时，该节点根据反馈调整搜索关键词、清洗候选池或触发重新拉取。

格式化输出节点

该节点生成最终推荐列表，并将推荐结果交给前端展示。

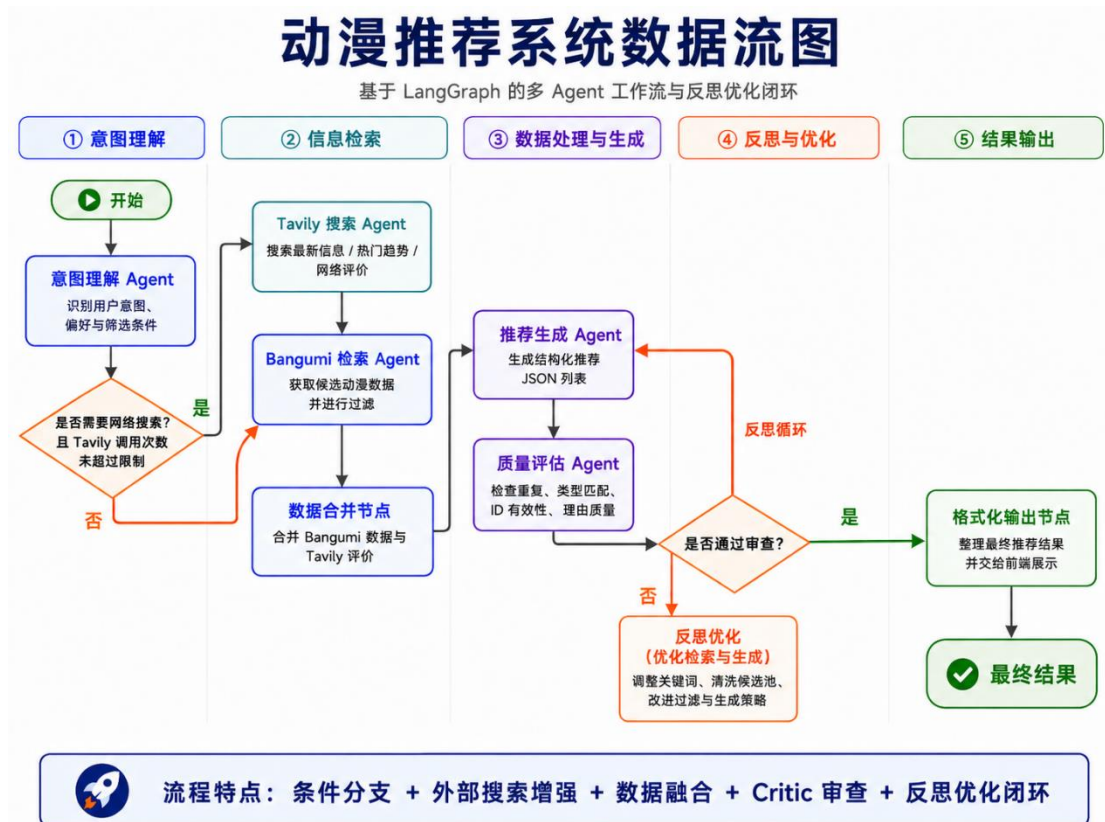


图 3.2: Agent 交互流程图

3.3 数据流设计

系统数据流可以分为五个阶段。

第一阶段是输入阶段。用户输入自然语言需求，同时可在侧边栏选择类型、年代、最低评分和标签。系统将这些信息封装为初始 `AgentState`。

第二阶段是意图结构化阶段。系统通过大模型将用户输入转换为结构化字段，例如搜索关键词、浏览标签、偏好信息和过滤参数。

第三阶段是候选数据获取阶段。系统调用 `Bangumi API` 获取候选作品，必要时调用 `Tavily` 获取网络评价。`Bangumi` 数据是主要事实来源，`Tavily` 数据是辅助参考。

第四阶段是推荐生成与质量控制阶段。系统从候选池中选择作品并生成推荐理由，然后通过 `Critic` 节点审查结果。如果不通过，则进入反思循环。

第五阶段是结果展示阶段。系统将最终推荐结果交给 `Streamlit` 前端，以卡片形式展示作品封面、标题、评分、标签、推荐理由和 `Bangumi` 链接。

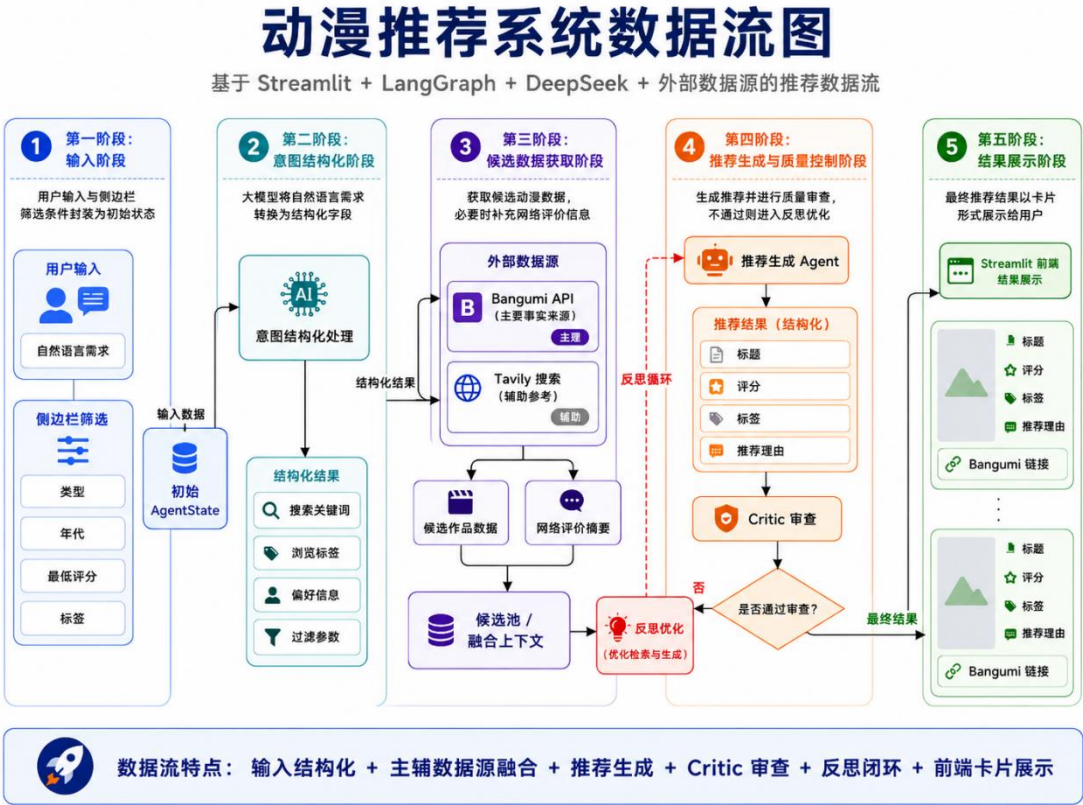


图 3.3：系统数据流图

四、关键实现与代码展示

4.1 LangGraph 核心循环设计

本项目采用 LangGraph 构建多智能体工作流，其核心特点是支持条件分支和反思循环，而不是简单的线性执行流程。整个流程由多个功能节点组成，包括意图理解、网络搜索、Bangumi 数据获取、推荐生成、质量评估和结果格式化等模块。

```
def build_graph():  
    builder = StateGraph(AgentState)  
  
    builder.add_node("understand_intent", understand_intent)  
    builder.add_node("search_tavily", search_tavily_node)  
    builder.add_node("fetch_bangumi", fetch_bangumi)  
    builder.add_node("merge_data", merge_data)  
    builder.add_node("generate_recommendations", generate_recommendations)  
    builder.add_node("evaluate_recommendations", evaluate_recommendations)  
    builder.add_node("refine_search", refine_search)  
    builder.add_node("format_response", format_response)  
  
    builder.add_edge(START, "understand_intent")  
  
    builder.add_conditional_edges(  
        "understand_intent",  
        decide_search,  
        {  
            "parallel": "search_tavily",  
            "direct": "fetch_bangumi",  
        },  
    ),
```

```
)

builder.add_edge("search_tavily", "fetch_bangumi")
builder.add_edge("fetch_bangumi", "merge_data")
builder.add_edge("merge_data", "generate_recommendations")
builder.add_edge("generate_recommendations",
"evaluate_recommendations")

builder.add_conditional_edges(
    "evaluate_recommendations",
    decide_reflection,
    {
        "refine": "refine_search",
        "format": "format_response",
    },
)

builder.add_conditional_edges(
    "refine_search",
    decide_refetch,
    {
        "refetch": "fetch_bangumi",
        "recommend": "generate_recommendations",
    },
)

builder.add_edge("format_response", END)
return builder.compile()
```

具体执行过程如下：

系统首先进入意图理解节点，对用户输入进行分析，提取搜索关键词、用户偏好以及筛选条件。

根据分析结果判断是否需要调用 Tavily 获取网络热度或时效性信息。如果需要，则先执行网络搜索；否则直接进入 Bangumi 数据检索阶段。

系统从 Bangumi 获取候选动漫数据，并与网络搜索结果进行整合，形成统一的候选池。

推荐生成节点基于候选池和用户偏好生成初步推荐结果。

推荐结果随后进入 Critic 节点进行质量评估，检查推荐是否满足用户需求、是否存在重复推荐以及是否违反筛选条件。

如果评估通过，则直接输出最终结果；如果评估未通过，则进入搜索优化节点，对搜索策略或候选数据进行调整，并重新执行推荐流程。

整个工作流形成了“理解需求—检索数据—生成推荐—质量评估—反思优化”的闭环结构。相比传统的一次性生成方式，这种设计能够有效降低大模型幻觉带来的影响，提高推荐结果的准确性和稳定性，同时增强系统的可解释性和鲁棒性。

4.2 状态设计 AgentState

系统通过统一的 AgentState 在各个节点之间传递信息。为了便于不同 Agent 之间共享上下文信息，项目使用 TypedDict 定义统一状态结构。下面给出简化后的状态定义，并对主要字段进行说明：

```
class AgentState(TypedDict):  
    messages: list                # 对话历史消息  
    user_query: str               # 用户原始输入  
    preferences: dict             # 从用户输入中提取的偏好信息  
    filter_params: dict            # 类型、评分、年代等筛选条件  
    search_keywords: str          # 用于 Bangumi 搜索的核心关键词  
    browse_tags: list             # 用于标签浏览的关键词列表  
    need_tavily: bool             # 是否需要调用 Tavily 搜索
```

tavily_results: str	# Tavily 返回的网络搜索摘要
bangumi_data: list	# Bangumi 返回的候选动漫数据
candidate_recommendations: list	# 大模型生成的候选推荐结果
final_recommendations: list	# 最终输出给用户的推荐结果
reflection_count: int	# 当前反思/重试次数
tavily_call_count: int	# Tavily 已调用次数
pass_evaluation: bool	# 是否通过质量评估
evaluation_feedback: str	# Critic 节点给出的反馈意见
need_refetch: bool	# 是否需要重新检索数据
execution_path: list	# Agent 实际执行路径记录
process_log: list	# 系统运行过程日志

其中，`user_query`、`preferences` 和 `filter_params` 主要用于保存用户需求；`search_keywords`、`tavily_results` 和 `bangumi_data` 用于保存检索阶段产生的数据；`candidate_recommendations`、`final_recommendations` 和 `evaluation_feedback` 用于保存推荐与评估结果；`reflection_count`、`execution_path` 和 `process_log` 则用于记录系统执行过程，便于调试和分析。

状态设计的好处是每个节点只需要读取自己需要的字段，并返回需要更新的字段。这样可以避免不同模块之间直接互相调用，降低模块耦合度，提升系统的可维护性和扩展性。

4.3 意图理解实现

意图理解节点负责将用户自然语言转换成结构化数据。其核心任务包括：

提取 Bangumi 搜索关键词。

提取 Bangumi 标签浏览词。

判断是否需要 Tavily 网络搜索。

提取用户偏好，例如喜欢的类型、排除的类型、情绪倾向、风格偏好和年代偏好。

解析评分约束，例如“高分”“8 分以上”“神作”等。

识别番剧类型，例如 TV、剧场版等。

该节点的设计重点在于避免把“高分”“推荐”“番剧”等通用词直接作为搜索关键词，因为这类词在 Bangumi 中容易导致搜索偏差。系统会将这些词转换为过滤参数，而不是作为关键词查询。

4.4 Bangumi 工具定义

Bangumi 工具主要包括关键词搜索、标签浏览、详情获取和数据过滤。简化后的工具逻辑如下：

```
def search_bangumi(keyword):  
    if not keyword:  
        return []  
    results = do_search(keyword)  
    if results:  
        return results  
  
    # 如果组合关键词搜不到，则逐步缩短关键词  
    words = keyword.split()  
    for i in range(len(words) - 1, 0, -1):  
        shorter = " ".join(words[:i])  
        results = do_search(shorter)  
        if results:  
            return results  
  
    return []
```

系统还实现了多关键词搜索和去重机制。例如当用户输入“异世界 搞笑”时，系统不仅会搜索组合词，也会分别搜索“异世界”和“搞笑”，再根据条目 ID 进行合并去重。

4.5 数据过滤实现

系统对 Bangumi 数据进行多层过滤：

类型过滤

根据 TV、剧场版、OVA、WEB 等类型筛选。

年代过滤

根据播出年份范围筛选。

评分过滤

根据用户设置或自然语言中解析出的最低评分筛选。

标签过滤

根据用户选择的标签或系统解析出的标签筛选。

低质量过滤

剔除明显不是正规番剧的条目，例如名称中包含“毕设”“答辩”“课程设计”“论文”等关键词的条目。

简化后的过滤逻辑如下：

```
def filter_bangumi_data(data, filter_params):  
    filtered = []  
  
    for item in data:  
        if type_not_match(item, filter_params):  
            continue  
        if year_not_match(item, filter_params):  
            continue  
        if score_not_match(item, filter_params):  
            continue  
        if tags_not_match(item, filter_params):  
            continue  
  
        filtered.append(item)  
  
    return filtered
```

这一部分是系统可靠性的重要保障。如果没有数据过滤，推荐结果中可能混入不相关作品、低质量条目或不符合用户筛选条件的作品。

4.6 Tavily 多维搜索实现

Tavily 搜索主要用于补充网络评价和时效性信息。当用户输入“最新”“热门”“今年”“网络口碑”等词语时，系统会构建多条搜索查询，例如：

```
queries = [  
    f'{base} 动漫 推荐 最佳',  
    f'{base} 动漫 评价 口碑',  
    f'{base} anime review recommendation rating',  
    f'{base} anime 人气排名 热门推荐 2024 2025',  
]
```

系统会对搜索结果进行去重，并按照关键词、标签、评价词命中情况计算相关性，最后将精选结果整理为摘要，提供给推荐生成 Agent 作为辅助上下文。

4.7 推荐生成实现

推荐生成 Agent 的输入包括用户查询、用户偏好、Bangumi 候选数据和 Tavily 网络评价。系统要求大模型输出严格 JSON 格式，例如：

```
[  
    {  
        "title": "作品中文名",  
        "title_original": "作品原名",  
        "reason": "推荐理由",  
        "match_points": ["匹配点 1", "匹配点 2"],  
        "score": 8.5,  
        "tags": ["恋爱", "校园"],  
        "image_url": "封面链接",  
        "bangumi_id": 12345  
    }  
]
```


]

为了降低大模型幻觉，系统不是让模型凭空推荐，而是将候选池传给模型，让模型只能从已有候选作品中选择。同时系统要求 `bangumi_id` 必须是有效正整数，避免输出不存在的条目。

4.8 质量评估与反思循环

质量评估 Agent 扮演 Critic 角色，主要检查以下问题：

是否存在同一系列多部作品重复推荐。

每部作品的核心类型是否匹配用户查询。

如果用户提到特定作品，推荐是否与该作品相似。

推荐理由是否具体，是否包含与用户查询相关的匹配点。

如果评估不通过，系统进入 `refine_search` 节点。该节点会根据反馈调整搜索关键词、清洗候选池或触发重新拉取。系统最多进行两次反思，避免无限循环。

反思循环是本项目区别于普通推荐系统的重要设计。它使系统具有“生成—评估—修正”的闭环能力，提高了推荐结果的稳定性。

4.9 配置文件设计

系统通过 `.env` 文件配置 API Key，避免将密钥硬编码到代码中。配置项包括：

`DEEPSEEK_API_KEY=你的 DeepSeek API 密钥`

`TAVILY_API_KEY=你的 Tavily API 密钥`

依赖文件 `requirements.txt` 中包含：

`langgraph`

`langchain`

`langchain-openai`

`streamlit`

`tavily-python`

`requests`

`python-dotenv`

pydantic

这种配置方式便于部署和迁移，也符合基本的软件工程安全规范。

4.10 AI IDE 使用说明

在项目开发过程中，使用 Trae CN 辅助完成以下工作：

根据课程需求生成初始项目结构。

编写 LangGraph 节点和状态定义。

设计 prompt，让大模型输出稳定 JSON。

优化 Bangumi 搜索、过滤和高质量条目识别逻辑。

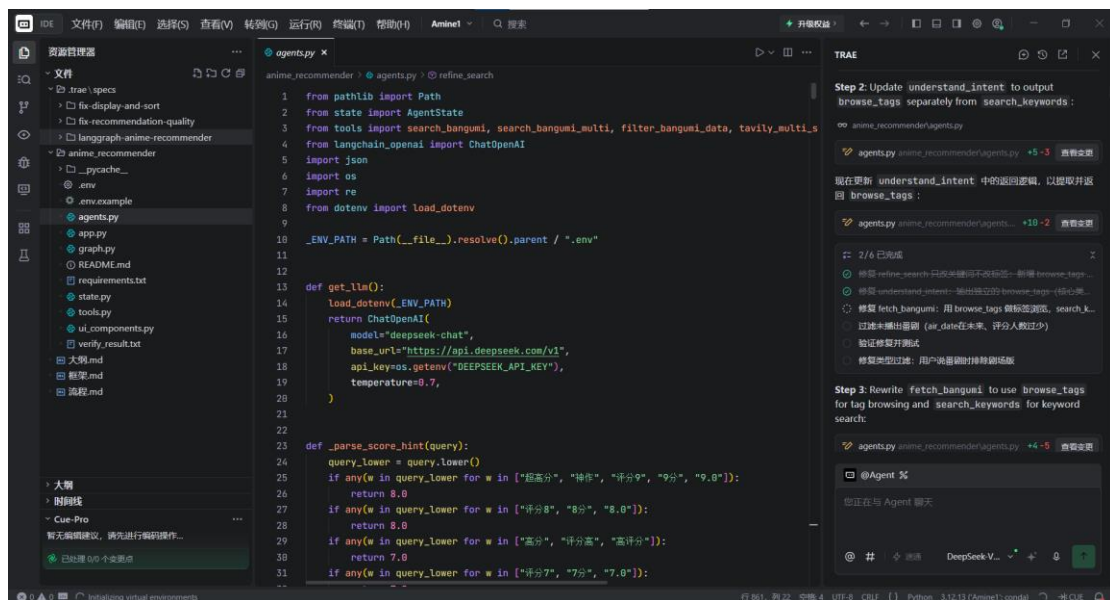


图 4.1: Trae CN 使用截图

五、测试与评估

5.1 功能测试

系统测试主要围绕以下功能展开：

类型识别测试

测试系统是否能够根据作品名称和集数判断作品类型。例如集数为 12 的作品应识别为 TV，集数为 1 且名称包含剧场版的作品应识别为剧场版，名称包含 OVA 的作品应识别为 OVA。

低质量条目过滤测试

测试系统是否能够识别并过滤“毕设答辩”“毕业设计”等非正规番剧条目。

TV 类型过滤测试

输入包含 TV 和剧场版的候选数据，筛选 TV 后应只保留 TV 类型作品。

Bangumi 端到端搜索测试

测试系统能否通过关键词调用 Bangumi 搜索，并返回包含作品名称、评分、类型和收藏数的数据。

真实数据过滤测试

对 Bangumi 返回的真实数据进行 TV 类型过滤，验证过滤逻辑是否有效。

Graph 编译测试

测试 LangGraph 状态图能否成功编译，确保节点和边的定义没有结构性错误。

5.2 测试结果分析

根据项目测试结果，系统完成了以下验证：

第一，类型识别功能可以正常工作。系统能够将常见 TV 动画、剧场版、OVA 和未知类型条目映射到对应类型。

第二，低质量检测功能有效。正常条目不会被误判，而包含“毕设答辩”“毕业设计”等关键词的条目会被识别为低质量条目。

第三，类型过滤功能有效。输入 1 个 TV 条目和 1 个剧场版条目后，经过 TV 过滤只保留 1 个 TV 条目。

第四, Bangumi 端到端搜索可以返回真实搜索结果, 并且测试中能够发现低质量条目, 说明低质量过滤模块确实有应用价值。

第五, LangGraph 能够成功编译, 说明 Agent 节点和状态流转结构基本正确。

5.3 Agent 行为评估

Agent 行为评估主要从以下几个方面进行:

意图理解准确性

观察系统是否能从用户查询中提取正确关键词。例如“高分恋爱番剧”应提取“恋爱”作为核心搜索词, 并将“高分”转化为评分约束。

工具调用合理性

当用户查询包含“最近”“今年”“热门”等词时, 系统应调用 Tavily; 当用户只是普通类型推荐时, 系统应直接调用 Bangumi, 避免不必要的网络搜索。

推荐结果相关性

推荐作品的核心类型应与用户需求一致。例如用户想要“恋爱喜剧”, 系统不应推荐核心为机战、体育或科幻的作品。

反思修正能力

当 Critic 判断推荐不通过时, 系统应根据反馈调整关键词或候选池, 而不是直接输出错误结果。

可解释性

每部推荐都应包含推荐理由和匹配点, 用户能够理解为什么推荐该作品。

5.4 Demo 展示设计

Demo 选择以下三个场景:

场景一: 普通推荐

输入: “推荐几部高分恋爱喜剧番。”

预期: 系统直接通过 Bangumi 检索获取候选作品, 并返回恋爱/喜剧相关作品。根据前面的功能测试和 Agent 行为评估结果, 系统能够正确识别“恋爱”为核心搜索关键词, 并将“高分”转换为评分约束条件。在实际运行中, 推荐结果能够较

好地满足用户需求，推荐作品的核心标签与用户查询保持一致，同时推荐理由中能够明确说明作品与“恋爱喜剧”需求之间的匹配关系。



推荐几部高分恋爱喜剧番



为您找到 5 部推荐动漫，详细如下：



辉夜大小姐想让我告白-超级浪漫-

かぐや様は告らせたい-ウルトラロマンティック-

★ 8.3/10

恋爱

搞笑

校园

漫画改

日常

 该作是恋爱喜剧类型的标杆作品，第三季在保持高水准搞笑的同时，恋爱主线大幅推进，口碑和观看体验极佳。

匹配点:

- 核心类型为恋爱与搞笑，完美匹配用户对恋爱喜剧的需求
- 评分8.3，收藏数高达45042，是高分高人气作品

 [Bangumi 条目](#)

图 5.1：场景一结果展示

27



我心里危险的东西 第二季

僕の心のヤバイやつ 第2期

★ 8.3/10

恋爱 校园 纯爱 日常

📖 近年备受好评的纯爱喜剧，第二季在甜蜜互动和搞笑细节上更进一步，主角间的化学反应非常动人。

匹配点:

- 核心类型为恋爱与校园喜剧，精准匹配用户需求
- 评分8.3且收藏近3万，口碑与热度俱佳，氛围欢乐治愈

 [Bangumi 条目](#)

图 5.2: 场景一结果展示



图 5.3: 场景一工作流程展示

场景二：复杂约束推荐

输入：“想看 2010 年之后的治愈日常番，评分高一点，不要太沉重。”

预期：系统识别年代、评分和风格偏好，推荐轻松治愈类作品。结合 5.1 和 5.2 中的测试结果可以发现，系统的类型过滤、评分过滤和年代过滤功能均能够正常工作，能够有效剔除不满足条件的候选作品。在实际推荐过程中，系统能够正确识别“2010 年之后”“高评分”“治愈”“不要太沉重”等约束条件，并通过 Critic 节点进一步检查推荐结果是否符合要求，从而保证最终输出的作品在类型、年代和风格上均与用户需求保持较高一致性。

 想看 2010 年之后的治愈日常番，评分高一点，不要太沉重。

 为您找到 5 部推荐动漫，详细如下：



白箱

SHIROBAKO

★ 8.7/10

P.A.WORKS 原创 励志 白箱 2014年10月 水岛努 动画制作 TV

 以动画制作为背景的职场日常番，整体氛围积极向上，充满励志与治愈感，评分高且收藏量大。

图 5.4：场景二结果展示



日常

日常

★ 8.5/10

日常 京阿尼 搞笑 治愈 爆笑 电波 2011年4月 TV

 京阿尼经典搞笑治愈番，以无厘头日常为核心，轻松幽默，评分高且人气旺盛。

匹配点：

- 2011年播出，符合2010年后的时间要求
- 核心类型为日常与搞笑，无沉重元素，评分8.5，完美契合轻松治愈需求

 [Bangumi 条目](#)

图 5.5：场景二结果展示

智能体工作流程

 **标签解析与意图理解：**从查询'想看 2010 年之后的治愈日常番，评分高一点，不要太沉重。'中提取关键词'治愈 日常'，标签['治愈','日常'] |  评分约束: ≥ 7.0 (从查询中解析)

 **偏好提取：**["治愈", "日常"]

 **网络评价搜索：**否 (已用0次)

 **Bangumi数据库搜索：**标签浏览30部(标签=['治愈','日常']) + 关键词搜索25部 → 合并去重后51部

 **低质量过滤：**51→45部 (剔除学生作品等)

 **硬性条件筛选：**类型=tv, 年代=(2000, 2026), 评分 ≥ 7.0 → 17部

 **搜索策略优化：**Bangumi数据17部 + Tavily无网络评价

 **AI推荐生成：**从17部候选中筛选

 **处理步骤：**《CLANNAD ～AFTER STORY～》、《3月的狮子 第二季》、《水星领航员 第三季》、《白箱》、《蜂蜜与四叶草II》、《日常》、《监禁》、《奶龙和暴暴龙》、《白兔糖》、《相合之物》... 等17部

 **质量偏好：**评分 ≥ 6.0 + 高分优先

 **推荐输出：**5部

 **质量审查：** 通过 - 推荐列表符合用户查询意图。所有作品均为2010年后播出，核心类型为治愈/日常，无沉重情节，评分均在7.7以上，满足高分要求。推荐理由均包含与用户需求相关的匹配点。未发现同一系列多部作品重复的情况，核心类型匹配无误。整体推荐合理，通过审查。

 **最终输出：**5部推荐结果

图 5.6：场景二工作流程展示

场景三：时效推荐

输入：“最近比较热门的异世界番有哪些？”

预期：系统调用 Tavily 搜索网络热度，同时调用 Bangumi 获取结构化数据。测试结果表明，当查询中包含“最近”“热门”等时效性表达时，系统能够正确触发 Tavily 工具调用，并将网络搜索结果与 Bangumi 数据进行融合。最终推荐结果不仅包含作品评分、标签等结构化信息，还能够结合网络口碑和近期热度给出推荐理由，体现出多工具协同带来的优势。

最近比较热门的异世界番有哪些？

为您找到 5 部推荐动漫，详细如下：



葬送的芙莉莲

葬送のフリーレン

★ 8.5/10

治愈 公路片 奇幻 异世界 2023年10月

2023年10月开播，是近期最热门的异世界题材作品之一，以独特的寿命论视角在异世界旅行，口碑极高。

匹配点:

- 核心设定为异世界奇幻冒险，符合用户对异世界番的需求

图 5.7：场景三结果展示



无职转生～到了异世界就拿出真本事～ 第2部分

無職転生 ～異世界行ったら本気だす～ 第2クール

★ 8.3/10

异世界 轻小说改 奇幻 冒险 2021年10月

异世界穿越题材的标杆作品，2021年10月播出，制作精良，剧情扎实，是近年异世界番的代表作。

匹配点:

- 核心类型为异世界穿越，完全符合用户查询
- 2021年播出，属于近期热门异世界作品
- 评分8.3，收藏数45378，口碑与热度兼备

[Bangumi 条目](#)

图 5.8：场景三结果展示



图 5.9: 场景三工作流程展示

六、系统升级与扩展

6.1 可扩展架构

本项目具有较好的可扩展性，主要体现在以下方面：

第一，Agent 节点可扩展。当前系统包含意图理解、搜索、推荐、评估和优化节点，未来可以继续增加用户画像 Agent、长期记忆 Agent、多轮对话澄清 Agent、召回排序 Agent 等。

第二，工具可扩展。当前系统接入 Bangumi 和 Tavily，未来可以接入 MyAnimeList、AniList、豆瓣、Bilibili 番剧、维基百科等数据源。

第三，评估机制可扩展。当前 Critic 主要基于大模型审查，未来可以加入规则评估、统计指标评估和人工反馈评估。

第四，前端可扩展。当前使用 Streamlit 快速构建界面，未来可以迁移到 Vue、React 或移动端小程序。

第五，数据存储可扩展。当前系统主要实时调用外部 API，未来可以加入本地数据库、缓存系统和向量数据库，提高响应速度和个性化能力。

6.2 下一阶段计划

下一阶段可以从以下几个方向进行改进：

增加用户画像

记录用户喜欢和不喜欢的作品、类型、评分偏好和观看历史，实现个性化推荐。

增加多轮澄清

当用户需求过于模糊时，系统可以主动提问，例如“你更喜欢轻松搞笑还是剧情深刻？”“是否接受老番？”这样可以提高推荐准确率。

引入 RAG 和向量检索

将动漫简介、评论、标签和剧情关键词向量化，支持“类似某部作品”“氛围相近”“主角成长型”等语义检索。

增加缓存机制

对 Bangumi 搜索结果、Tavily 搜索结果和热门推荐结果进行缓存，减少重复

API 调用，提高响应速度。

加强评估指标

增加自动化 Benchmark，对推荐相关性、过滤条件满足率、无效 ID 率和重复推荐率进行量化评估。

增加用户反馈闭环

用户可以对推荐结果点赞、点踩或标记“不感兴趣”，系统根据反馈优化后续推荐。

增加多模态能力

未来可以分析动漫封面、PV 截图或海报风格，实现视觉风格推荐，例如“画风像新海诚”“想看色彩明亮的治愈番”。

6.3 AI 能力演进路径

本项目当前主要使用大模型进行文本理解和生成，未来 AI 能力可以分三个阶段演进。

第一阶段：LLM + 工具调用

系统通过大模型理解用户需求，通过 API 工具获取数据，并生成推荐理由。这是当前项目已经实现的阶段。

第二阶段：LLM + RAG + 用户画像

系统引入向量数据库和用户画像，使推荐结果不仅来自实时搜索，还能结合用户历史偏好和语义相似度。

第三阶段：多智能体协作 + 多模态推荐

系统进一步引入多个专业 Agent，例如剧情分析 Agent、评论总结 Agent、视觉风格 Agent、用户偏好 Agent 和排序 Agent，实现更复杂、更个性化的推荐。

七、课程总结

7.1 个人收获

本课程最大的收获，是让我真正体验了一次以大模型为核心的完整开发流程。课程本身鼓励使用 AI 辅助编程，因此在本项目开发过程中，我大量使用了 ChatGPT、DeepSeek 等 AI 工具参与代码生成、方案设计和问题排查。最初我担心这种方式会削弱学习效果，但在实际开发中，我发现恰恰相反：AI 更像是一位高效率的协作伙伴，而不是替代开发者本身。

在项目实现过程中，虽然大量代码由 AI 生成，但系统架构设计、技术路线选择、模块划分、Prompt 调整、工具接入以及最终效果验证仍然需要开发者自己完成。例如，如何将系统拆分为意图理解、数据检索、推荐生成、质量评估和反思优化等多个节点，如何设计状态流转方式，如何处理 Bangumi 数据质量问题，以及如何让各模块协同工作，这些都需要不断思考、实验和迭代。

通过这一过程，我对大模型应用开发有了更系统的认识。过去使用大模型时，更多是把它当作一个“问答工具”；而在本项目中，我逐渐认识到，大模型更适合作为系统中的一个智能决策模块，需要与外部工具、状态管理、流程编排和反馈机制结合起来，才能形成真正可用的应用系统。

此外，本项目也让我理解了多智能体系统的基本工程思路。复杂任务不应该全部交给一个模型一次性完成，而应该拆分为多个职责清晰的节点协同完成。这种设计不仅提高了系统稳定性，也使调试和扩展更加容易。

7.2 工程思维转变

本项目带来的最大转变，是从“编写代码”转向“设计系统”。

在 AI 编程环境下，代码本身已经不再是最稀缺的资源。借助 AI 工具，很多基础代码都可以快速生成，但系统是否真正可用，取决于开发者能否提出正确的问题、设计合理的架构，并持续验证和优化结果。

项目初期，我更多关注的是如何让系统运行起来；但随着开发深入，我逐渐意识到，仅仅能够输出结果是不够的，还需要考虑结果是否可靠、是否满足用户

需求、是否能够解释、失败时如何兜底、模块之间如何解耦，以及后续如何扩展。这些问题体现的正是工程化思维。

例如，系统中加入低质量过滤规则，是因为真实数据中可能混入“毕设答辩”“课程设计”等非正规条目；加入 Critic 节点，是因为大模型第一次生成的结果并不一定完全符合要求，需要自动审查和修正机制；将 API Key 放入 .env 文件，则体现了工程项目对安全性和部署便利性的考虑。

同时，在使用 AI 辅助开发时，我也认识到不能完全依赖 AI 输出。AI 生成的代码可能存在逻辑错误、接口调用错误或不符合项目需求的情况，因此必须通过阅读代码、调试运行和查阅文档来验证其正确性。只有真正理解系统设计和实现原理，才能有效利用 AI 提高开发效率。

因此，本项目不仅是一个动漫推荐应用，更是一次人与 AI 协同开发 Agent 系统的实践。它让我认识到，大模型时代的软件开发能力，不仅包括编码能力，更包括系统设计能力、问题拆解能力以及对 AI 输出进行判断和修正的能力。

7.3 对课程的建议

总体而言，我认为课程设计非常符合当前大模型应用的发展趋势，尤其是鼓励学生使用 AI 辅助开发这一点，使课程更贴近真实行业实践。

在此基础上，我认为课程还可以进一步加强以下几个方面。

第一，可以增加工程评估环节。大模型应用不仅要看是否能够运行，还应该评估准确率、稳定性、响应速度、用户体验以及失败案例，这样更接近真实企业开发过程。

第二，可以增加部署相关内容。例如如何将应用部署到服务器，如何配置环境变量，如何管理 API Key，如何记录日志以及如何处理并发请求等。

第三，可以继续鼓励学生使用 AIIDE 或大模型辅助开发，同时要求学生能够解释系统架构、核心模块设计以及关键代码逻辑。这样既能够提高开发效率，也能够保证学生真正理解项目实施过程。

7.4 总结

本项目完成了一个基于 LangGraph 的多智能体动漫推荐系统，实现了从自

然语言输入到结构化推荐输出的完整流程。系统结合 DeepSeek 大模型、Bangumi 动漫数据库、Tavily 网络搜索和 Streamlit 前端界面，具备意图理解、数据检索、硬性筛选、推荐生成、质量评估、反思优化和可视化展示等功能。

更重要的是，本项目让我体验了 AI 辅助开发模式下的软件工程实践。虽然项目中的大量代码借助 AI 工具生成，但整个开发过程仍然需要开发者负责需求分析、系统设计、模块组织、效果验证和持续优化。通过不断与 AI 协作、调试和迭代，我不仅掌握了 Agent workflow 设计、工具调用、状态管理、Prompt 约束和数据清洗等关键技术，也更加理解了大模型时代软件开发的新范式。

未来如果继续完善该系统，可以进一步加入用户画像、向量检索、多轮澄清、缓存机制和自动化评估等能力，使其从课程 Demo 逐步升级为更完整、更稳定、更个性化的智能推荐系统。